APACHE KAFKA

Rajesh Pasham

- Apache Kafka was originated at LinkedIn and later became an open sourced Apache project in 2011, then First-class Apache project in 2012.
- Kafka is written in Scala and Java.
- Apache Kafka is publish-subscribe based fault tolerant messaging system.
- It is fast, scalable and distributed by design.

- In comparison to other messaging systems, Kafka has better throughput, built-in partitioning, replication and inherent fault-tolerance, which makes it a good fit for large-scale message processing applications.

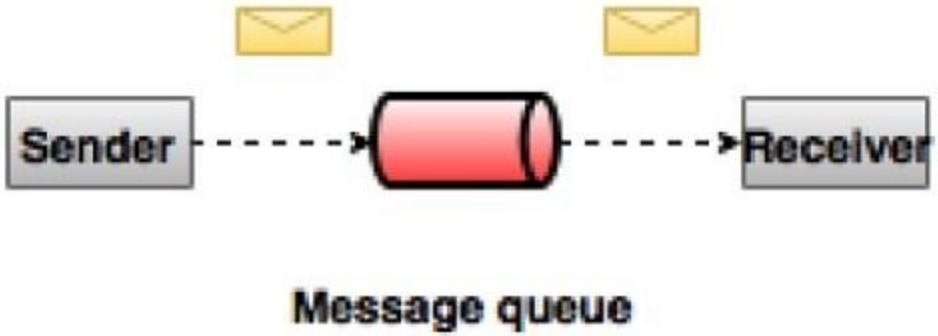
What is a Messaging System?

- A Messaging System is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it.
- Distributed messaging is based on the concept of reliable message queuing.
- Messages are queued asynchronously between client applications and messaging system.
- Two types of messaging patterns are available − one is point to point and the other is publish-subscribe (pub-sub) messaging system.

Point to Point Messaging System

- In a point-to-point system, messages are persisted in a queue.
- One or more consumers can consume the messages in the queue, but a particular message can be consumed by a maximum of one consumer only.
- Once a consumer reads a message in the queue, it disappears from that queue.
- The typical example of this system is an Order Processing System, where each order will be processed by one Order Processor, but Multiple Order Processors can work as well at the same time.

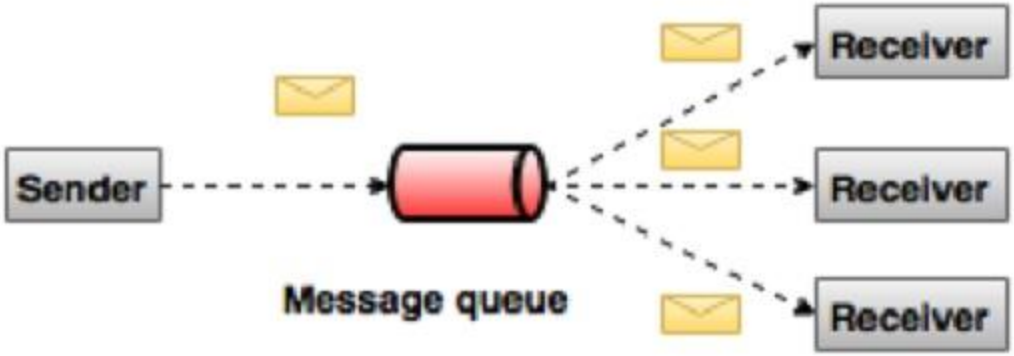
Point to Point Messaging System



Publish-Subscribe Messaging System

- In the publish-subscribe system, messages are persisted in a topic.
- Unlike point-to-point system, consumers can subscribe to one or more topic and consume all the messages in that topic.
- In the Publish-Subscribe system, message producers are called publishers and message consumers are called subscribers.
- A real-life example is Dish TV, which publishes different channels like sports, movies, music, etc.

Publish-Subscribe Messaging System



What is Kafka?

- Apache Kafka is a distributed publish-subscribe messaging system and a robust queue that can handle a high volume of data and enables you to pass messages from one end-point to another.

- Kafka is suitable for both offline and online message consumption.

- Kafka messages are persisted on the disk and replicated within the cluster to prevent data loss.

What is Kafka?

- Kafka is built on top of the ZooKeeper synchronization service.
- It integrates very well with Apache Storm and Spark for real-time streaming data analysis.

Benefits

- **Reliability** − Kafka is distributed, partitioned, replicated and fault tolerance.
- **Scalability** − Kafka messaging system scales easily without down time.
- **Durability** − Kafka uses Distributed commit log which means messages persists on disk as fast as possible, hence it is durable.
- **Performance** − Kafka has high throughput for both publishing and subscribing messages. It maintains stable performance even many TB of messages are stored.

Use Cases

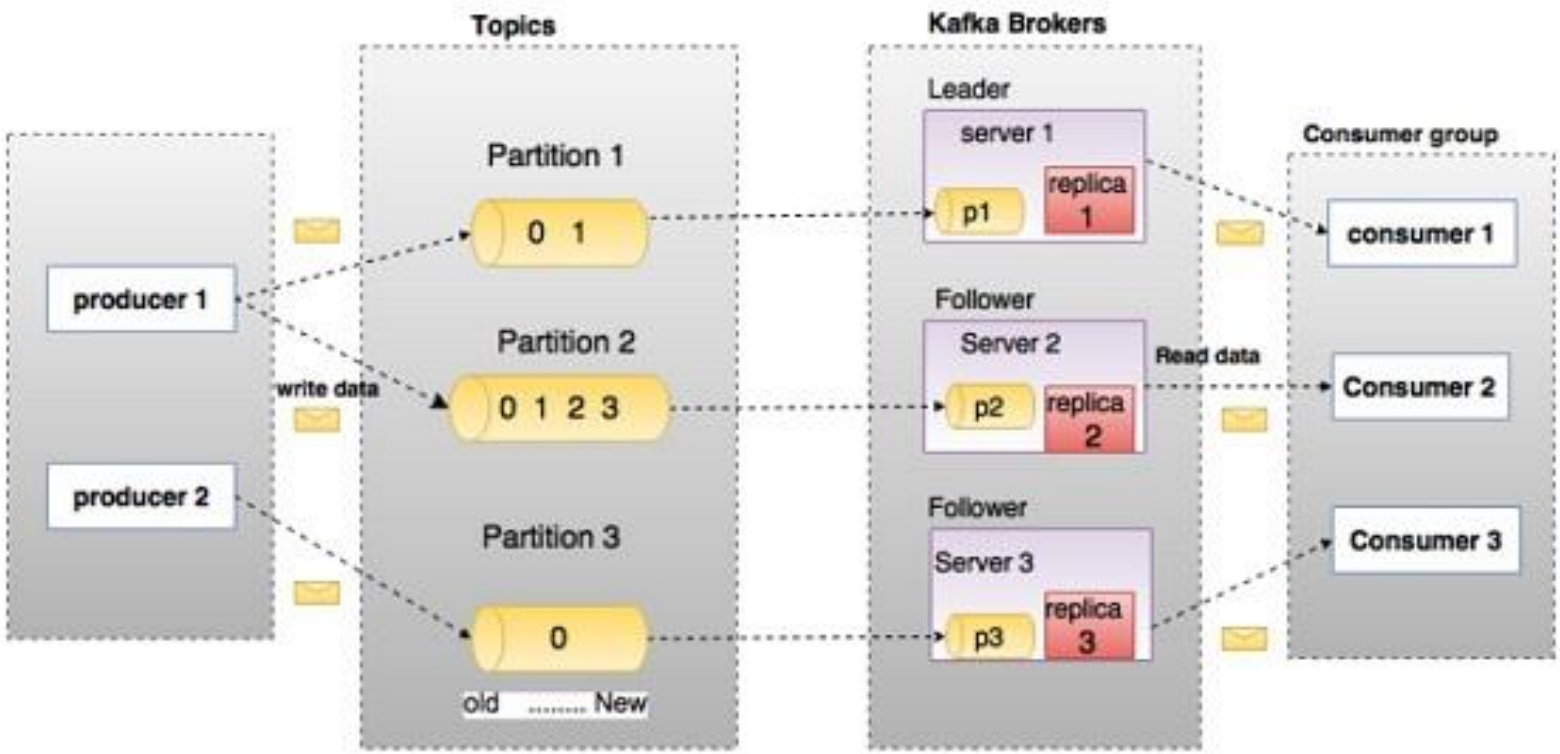
- **Metrics** − Kafka is often used for operational monitoring data.
- **Log Aggregation Solution** − Kafka can be used across an organization to collect logs from multiple services and make them available in a standard format to multiple consumers.
- **Stream Processing** − Popular frameworks such as Storm and Spark Streaming read data from a topic, processes it, and write processed data to a new topic where it becomes available for users and applications.

Need for Kafka

- Kafka is a unified platform for handling all the real-time data feeds.
- Kafka supports low latency message delivery and gives guarantee for fault tolerance in the presence of machine failures.
- It has the ability to handle a large number of diverse consumers.
- Kafka is very fast, performs 2 million writes/sec.

Apache Kafka - Fundamentals

- The following diagram illustrates the main terminologies



Apache Kafka - Fundamentals

- In the above diagram, a topic is configured into three partitions.
- Partition 1 has two offset factors 0 and 1.
- Partition 2 has four offset factors 0, 1, 2, and 3.
- Partition 3 has one offset factor 0.
- The id of the replica is same as the id of the server that hosts it.

Apache Kafka - Fundamentals

- Assume, if the replication factor of the topic is set to 3, then Kafka will create 3 identical replicas of each partition and place them in the cluster to make available for all its operations.

- To balance a load in cluster, each broker stores one or more of those partitions.

- Multiple producers and consumers can publish and retrieve messages at the same time.

Apache Kafka - Fundamentals

Topics

- A stream of messages belonging to a particular category is called a topic. Data is stored in topics. Topics are split into partitions.
- For each topic, Kafka keeps a minimum of one partition. Each such partition contains messages in an immutable ordered sequence.
- A partition is implemented as a set of segment files of equal sizes.

Apache Kafka - Fundamentals

Partition

- Topics may have many partitions, so it can handle an arbitrary amount of data.

Partition offset

- Each partitioned message has a unique sequence id called as offset.

Apache Kafka - Fundamentals

Replicas of partition

- Replicas are nothing but backups of a partition.
- Replicas are never read or write data.
- They are used to prevent data loss.

Apache Kafka - Fundamentals

Brokers

- Brokers are simple system responsible for maintaining the published data.
- Each broker may have zero or more partitions per topic.
- If there are N partitions in a topic and N number of brokers, each broker will have one partition.

Apache Kafka - Fundamentals

Brokers

- If there are N partitions in a topic and more than N brokers (n + m), the first N broker will have one partition and the next M broker will not have any partition for that particular topic.
- If there are N partitions in a topic and less than N brokers (n-m), each broker will have one or more partition sharing among them.
- This scenario is not recommended due to unequal load distribution among the broker.

Apache Kafka - Fundamentals

Kafka Cluster

- Kafka's having more than one broker are called as Kafka cluster.
- A Kafka cluster can be expanded without downtime.
- These clusters are used to manage the persistence and replication of message data.

Apache Kafka - Fundamentals

Producers

- Producers are the publisher of messages to one or more Kafka topics.
- Producers send data to Kafka brokers. Every time a producer publishes a message to a broker, the broker simply appends the message to the last segment file. Actually, the message will be appended to a partition.
- Producer can also send messages to a partition of their choice.

Apache Kafka - Fundamentals

Consumers

- Consumers read data from brokers.
- Consumers subscribes to one or more topics and consume published messages by pulling data from the brokers.

Leader

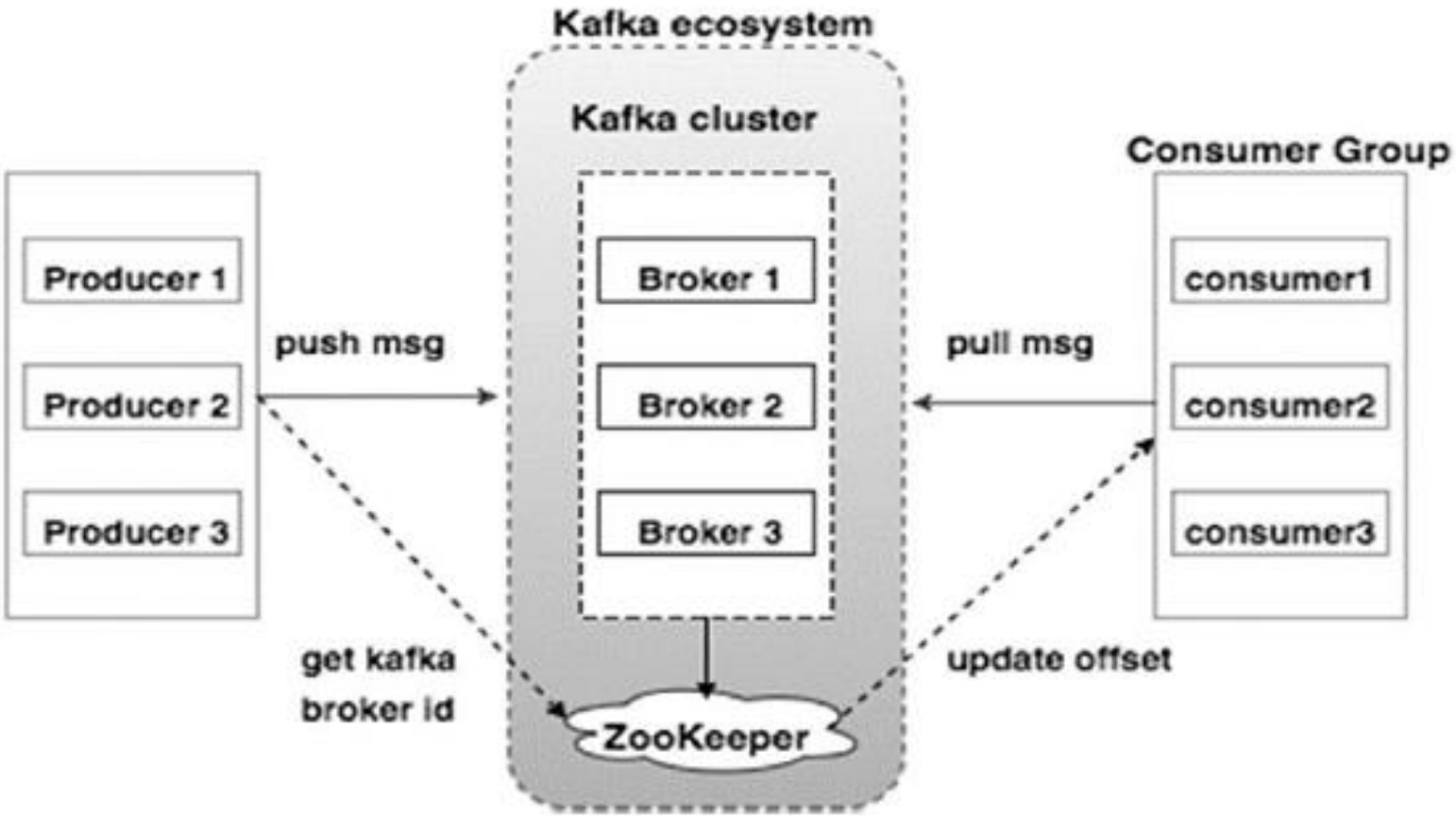
- Leader is the node responsible for all reads and writes for the given partition. Every partition has one server acting as a leader.

Apache Kafka - Fundamentals

Follower

- Node which follows leader instructions are called as follower.
- If the leader fails, one of the follower will automatically become the new leader.
- A follower acts as normal consumer, pulls messages and updates its own data store.

Apache Kafka - Cluster Architecture



Apache Kafka - Cluster Architecture

Broker

- Kafka cluster typically consists of multiple brokers to maintain load balance.
- Kafka brokers are stateless, so they use ZooKeeper for maintaining their cluster state.
- One Kafka broker instance can handle hundreds of thousands of reads and writes per second and each broker can handle TB of messages without performance impact.
- Kafka broker leader election can be done by ZooKeeper.

Apache Kafka - Cluster Architecture

ZooKeeper

- ZooKeeper is used for managing and coordinating Kafka broker.
- ZooKeeper service is mainly used to notify producer and consumer about the presence of any new broker in the Kafka system or failure of the broker in the Kafka system.
- As per the notification received by the Zookeeper regarding presence or failure of the broker then producer and consumer takes decision and starts coordinating their task with some other broker.

Apache Kafka - Cluster Architecture

Producers

- Producers push data to brokers.
- When the new broker is started, all the producers search it and automatically sends a message to that new broker.
- Kafka producer doesn’t wait for acknowledgements from the broker and sends messages as fast as the broker can handle.

Apache Kafka - Cluster Architecture

Consumers

- Since Kafka brokers are stateless, which means that the consumer has to maintain how many messages have been consumed by using partition offset.
- If the consumer acknowledges a particular message offset, it implies that the consumer has consumed all prior messages.
- The consumer issues an asynchronous pull request to the broker to have a buffer of bytes ready to consume.

Apache Kafka - Cluster Architecture

Consumers

- The consumers can rewind or skip to any point in a partition simply by supplying an offset value.
- Consumer offset value is notified by ZooKeeper.

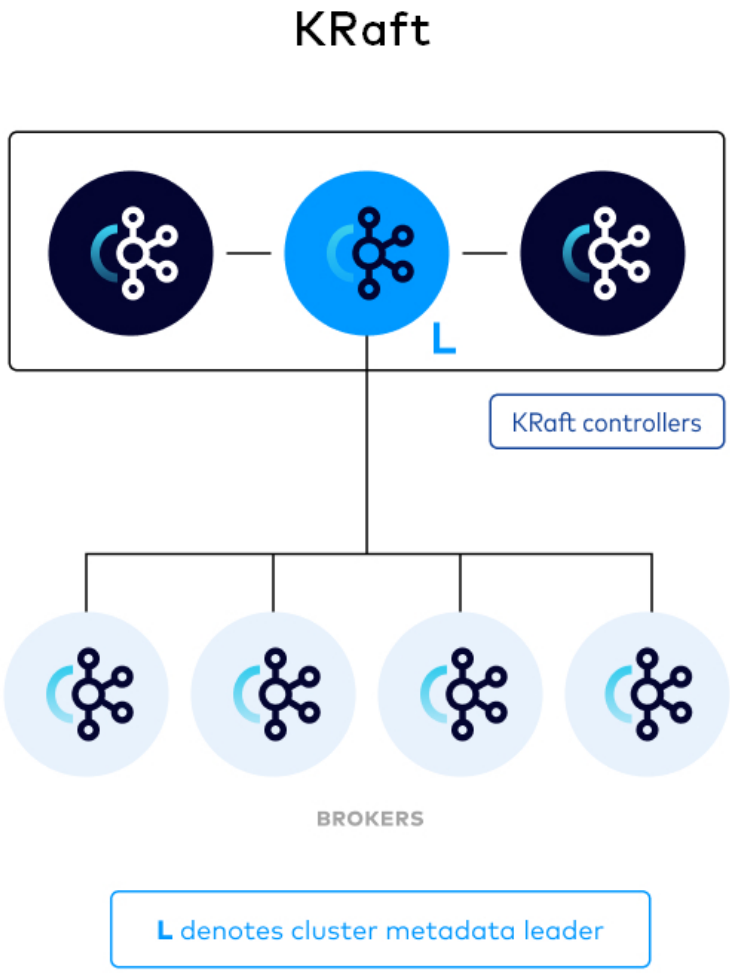
Introduction to KRaft in Kafka Platform

Simplifying Kafka Metadata Management

- **Kafka Raft (KRaft) Overview:** KRaft, short for Kafka Raft, is the consensus protocol that replaces ZooKeeper in Apache Kafka®. It consolidates metadata management directly into Kafka for improved simplicity and reliability.
- **Architectural Shift:** KRaft removes the external ZooKeeper dependency by embedding metadata management in Kafka itself. Each KRaft controller participates in a Raft quorum, ensuring strong consistency and streamlined control.
- **Motivation for Adoption:** The move to KRaft mode reduces operational complexity, enhances metadata propagation speed, and enables faster controller failover with fewer moving parts.
- **Key Benefit:** By eliminating ZooKeeper, Kafka simplifies deployment, increases fault tolerance, and supports larger Kafka clusters with lower maintenance overhead.

KRaft Architecture

How the Raft Quorum Manages Kafka Metadata



KRaft Architecture

How the Raft Quorum Manages Kafka Metadata

Controller Quorum and Metadata Log
KRaft controllers form a Raft quorum that manages the Kafka metadata log, storing all topic, partition, ISR, and configuration changes. This ensures consistency and durability across controllers.

Active and Follower Controllers
The active controller handles all broker RPCs, while follower controllers replicate metadata updates and serve as hot standbys, ensuring immediate recovery during failover.

Leader Election and Consensus
Using the Raft protocol, KRaft achieves consensus without external coordination. Majority voting enables leader election, fault tolerance, and fast recovery.

Snapshot and Recovery Mechanism
Controllers periodically write metadata snapshots to disk, improving recovery times and reducing replay overhead during restarts.

Scaling Kafka with KRaft

Performance, Partition Limits, and Failover Improvements

- **Partition and Cluster Limits:** Kafka scalability in KRaft mode depends on both the per-node partition count limit and the cluster-wide partition limit. Adding more nodes remains the primary way to increase capacity.
- **Enhanced Metadata Propagation:** KRaft's quorum controller minimizes metadata propagation delay, enabling faster synchronization across brokers and reducing cluster downtime.
- **Rapid Failover:** Experiments by Confluent demonstrate near-instantaneous controller failover in KRaft. In a 2-million-partition cluster, KRaft achieved 10× the partition scale of ZooKeeper-based clusters.
- **High Availability through Majority Consensus:** A three-node quorum can survive one controller failure; five nodes can survive two. Majority voting ensures operational continuity during failures.

KRaft Configuration Essentials

Core Settings Required for Confluent Platform Deployment

- **process.roles:** Specifies whether the Kafka server acts as a controller, broker, or both (though combined mode is not supported for production). This setting activates KRaft mode.
- **node.id:** Assigns a unique identifier to each node. No two servers—controller or broker—may share the same ID within a cluster.
- **controller.quorum.voters:** Defines the set of controllers participating in the Raft quorum using the format {id}@{host}:{port}. Ensures all nodes are aware of the quorum members.
- **controller.listener.names:** Lists controller listeners and ensures they match the listeners property. Essential for isolated or combined mode configurations.

Listeners, Logs, and Security in KRaft Mode

Ensuring Secure and Reliable Metadata Communication

- **Inter-Broker Communication:** Defined via either `inter.broker.listener.name` or `security.inter.broker.protocol`. These settings ensure secure communication between brokers and controllers.
- **Listener Configuration:** Controllers must list their own listeners under `controller.listener.names`, consistent with the quorum voters list. Brokers and controllers use mapped security protocols for communication.
- **Metadata Log Directories:** The `metadata.log.dir` property specifies where controller metadata is stored. If unset, it defaults to the first directory in `log.dirs`.
- **Security and Truststores:** Secure communication is achieved by defining SSL keystores and truststores per listener. Controllers must include truststore configurations for broker communication.

Tools for Managing and Debugging KRaft Mode

Command-Line Utilities for Cluster Insight and Diagnostics

kafka-metadata-quorum
Describes the runtime status of the KRaft metadata partition. Can connect via `--bootstrap-server` or `--bootstrap-controller` to inspect quorum state and leadership.

kafka-dump-log
Decodes and prints metadata records from KRaft log segments and snapshots, enabling deep inspection of stored metadata.

kafka-metadata-shell
Provides an interactive shell for exploring metadata partitions. Supports secure connections to running controllers via SSL.

kafka-migration-check
Assesses migration status and readiness for KRaft clusters. Useful for validating Confluent Platform transitions post-ZooKeeper.

Migration, Limitations & Best Practices

Transitioning from ZooKeeper and Ensuring KRaft Stability

Migration Process
Use the `kafka-migration-check` tool to validate migration readiness. Ensure each node's storage is formatted with a consistent cluster ID using the `kafka-storage format` command.

Unsupported Combined Mode
Running a Kafka node as both broker and controller (combined mode) is not supported in production due to security and operational gaps.

Quorum Reconfiguration
Prior to Confluent Platform 7.9, KRaft clusters could not add or remove controllers dynamically. Later versions introduce controlled quorum reconfiguration.

Monitoring Limitations
Certain tools, such as Health+, may still misreport KRaft controllers as brokers, impacting alert accuracy. Administrators should account for this discrepancy.

The Benefits of KRaft Architecture

Simpler, Faster, and More Reliable Kafka Metadata Management

- **Unified Metadata Management:** KRaft embeds metadata handling directly into Kafka, eliminating ZooKeeper and reducing operational overhead.
- **Improved Fault Tolerance:** Raft-based consensus and majority voting ensure resilient controller failover and minimal downtime.
- **Operational Efficiency:** Simplified configuration and deployment enable easier scaling and reduced administrative complexity.
- **Enhanced Scalability:** Supports up to 10× more partitions than ZooKeeper-based clusters, allowing massive deployments to operate efficiently.

Apache Kafka - Workflow

- Kafka is simply a collection of topics split into one or more partitions.
- A Kafka partition is a linearly ordered sequence of messages, where each message is identified by their index (called as offset).
- All the data in a Kafka cluster is the disjointed union of partitions.

Apache Kafka - Workflow

- Incoming messages are written at the end of a partition and messages are sequentially read by consumers.
- Durability is provided by replicating messages to different brokers.
- Kafka provides both pub-sub and queue based messaging system in a fast, reliable, persisted, fault-tolerance and zero downtime manner.

Apache Kafka - Workflow

- producers simply send the message to a topic and consumer can choose any one type of messaging system depending on their need.

Workflow of Pub-Sub Messaging

- Producers send message to a topic at regular intervals.
- Kafka broker stores all messages in the partitions configured for that particular topic.
- It ensures the messages are equally shared between partitions.
- If the producer sends two messages and there are two partitions, Kafka will store one message in the first partition and the second message in the second partition.

Workflow of Pub-Sub Messaging

- Consumer subscribes to a specific topic.
- Once the consumer subscribes to a topic, Kafka will provide the current offset of the topic to the consumer and also saves the offset in the Zookeeper ensemble.
- Consumer will request the Kafka in a regular interval (like 100 Ms) for new messages.
- Once Kafka receives the messages from producers, it forwards these messages to the consumers.

Workflow of Pub-Sub Messaging

- Consumer will receive the message and process it.
- Once the messages are processed, consumer will send an acknowledgement to the Kafka broker.
- Once Kafka receives an acknowledgement, it changes the offset to the new value and updates it in the Zookeeper.
- Since offsets are maintained in the Zookeeper, the consumer can read next message correctly even during server outages.

Workflow of Pub-Sub Messaging

- This above flow will repeat until the consumer stops the request.
- Consumer has the option to rewind/skip to the desired offset of a topic at any time and read all the subsequent messages.

Workflow of Queue Messaging / Consumer Group

- In a queue messaging system instead of a single consumer, a group of consumers having the same Group ID will subscribe to a topic.
- In simple terms, consumers subscribing to a topic with same Group ID are considered as a single group and the messages are shared among them.
- Producers send message to a topic in a regular interval.
- Kafka stores all messages in the partitions configured for that particular topic similar to the earlier scenario.

Workflow of Queue Messaging / Consumer Group

- A single consumer subscribes to a specific topic, assume Topic-01with Group ID as Group-1.
- Kafka interacts with the consumer in the same way as Pub-Sub Messaging until new consumer subscribes the same topic, Topic-01with the same Group ID as Group-1.
- Once the new consumer arrives, Kafka switches its operation to share mode and shares the data between the two consumers.

Workflow of Queue Messaging / Consumer Group

- This sharing will go on until the number of consumers reach the number of partition configured for that particular topic.

- Once the number of consumer exceeds the number of partitions, the new consumer will not receive any further message until any one of the existing consumer unsubscribes.

- This scenario arises because each consumer in Kafka will be assigned a minimum of one partition and once all the partitions are assigned to the existing consumers, the new consumers will have to wait.